

Deep learning and representation

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build Project 2, `nw/semantic`: a transformer encoder fine-tuned with LoRA for tags and priority at once, a training loop you can read, resumable checkpoints, an ONNX export with parity proof and an int8 variant, a similarity index of past tickets, and the benchmark that says exactly what the extra cost buys over Project 1.

Where we are

PART 1

Foundations and
the AI service layer

PART 2

Machine learning in
production

PART 3

Deep learning and
representation

PART 4

Generative AI:
retrieval and
evaluation

PART 5

Agentic AI systems

PART 6

Deployment,
observability and
capstone

Project 2 is measured against Project 1 on the same test split, the same machine. The retrieval project reuses its embedding model; the agent calls its service as a tool; the capstone routes between the two on the numbers you produce today.

Today, step by step

- 1 Environment, device, and the memory arithmetic
- 2 Read the loop and see the tests fail
- 3 Freeze the encoder, inject LoRA
- 4 The loss and the training step
- 5 Per-tag thresholds, then train the real model
- 6 Checkpoints that resume **OPTIONAL**
- 7 Export to ONNX, quantise, prove parity
- 8 The benchmark table
- 9 The similarity index and the service **OPTIONAL**
- 10 Error analysis on the German tickets **OPTIONAL**

The tests download nothing

They run a tiny randomly initialised DistilBERT with a two-thousand word tokenizer and finish in seconds on CPU. The real model comes in step 5.

Step 2 prints `2 failed, 5 passed, 8 errors`. That is the order of the next steps.

"It does not fit" is usually the optimiser, not the model

Held per parameter, fp32 with Adam	Bytes
Weights	4
Gradients	4
Two optimiser moments	8
Full fine-tuning	16

about 1 GB

66 million DistilBERT parameters at 16 bytes each, before a single activation

Activations scale with batch size times sequence length times hidden size times layers. Every number here is derivable; write it down before you touch a model.

Freeze the encoder and train under one percent of it

```
for p in model.encoder.parameters():
    p.requires_grad = False
cfg = LoraConfig(r=8, lora_alpha=16,
                 lora_dropout=0.05,
                 target_modules=["q_lin", "v_lin"],
                 bias="none")
model.encoder = get_peft_model(model.encoder, cfg)
```

Knob	Effect
Rank <code>r</code>	Adapter capacity. 8 for classification, 64 for generation
<code>alpha</code>	Scale of the adapter's contribution. Convention: 2 times <code>r</code>
Targets	Which linear layers get adapters. Attention query and value is the standard start
Dropout	Regularises the adapter; 0.05 by default

Small low-rank matrices on the query and value projections. Only those and the two heads train; the optimiser state becomes negligible, and a laptop CPU is enough.

Profile the input pipeline before you blame the accelerator

Fixed maximum length

Most of the compute goes to padding tokens. The accelerator is busy and nothing useful happens.

Dynamic padding

Pad to the longest ticket in the batch, not the maximum length. The single largest saving. The collate test proves it.

Workers

On a laptop, the main process is fine. On a GPU box, workers keep the device fed, and a deadlocked worker looks exactly like a slow GPU.

Tokenising to a fixed maximum length is the default in most tutorials, and it is the reason the accelerator looks busy while the loss barely moves per second.

Multi-label is not multi-class

	Tags	Priority
Can two be true at once?	Yes: "SSO" and "Outage"	No
Loss	Binary cross-entropy with logits, per tag	Cross-entropy with class weights
Decision	One threshold per tag, chosen on validation	Argmax
Wrong choice looks like	Softmax across tags: exactly one fires	A flat 0.5 under-predicts every rare tag

`loss_fn` adds the two. Priority is the triage project's imbalance lesson in a different loss.

Five lines per batch, and a finite check that names the step

- 1 Move `input_ids`, `attention_mask`, `tags` and `priority` to the device
- 2 Forward under `torch.autocast`, enabled only on CUDA; divide the loss by `accumulate` inside it
- 3 `scaler.scale(loss).backward()`
- 4 Every `accumulate` batches: unscale, clip to 1.0, step, update the scaler, zero the gradients, step the schedule
- 5 If the loss is not finite, raise with the step number. A NaN at step 40 is a learning rate problem and the message should say so

If the loss never moves: you divided by `accumulate` after backward instead of before, or `zero_grad` runs before the backward instead of after the step.

Read the seconds on the first loss line

0.3%

trainable parameters when `make train-semantic` prints its parameter line

- DistilBERT from the Hub, about 260 MB on the first run
- 2,000 ticket stratified subset, six epochs, about two minutes on Apple silicon
- Last epoch on validation: tag micro-F1 roughly 0.55, priority macro-F1 roughly 0.66, P0 recall above 0.8

If 20 steps took longer than a minute, switch to Colab now. A T4 replaces `make train-semantic` and `make export-semantic` ; unzip the download into `artifacts/semantic` and rejoin at step 8. Every test still runs locally.

A weights-only checkpoint makes the run after the crash a different run

As shipped

Weights only. Adam restarts cold and the learning rate schedule restarts from zero. The resume test fails because the optimiser has no moments.

What a resume needs

Adapter and head weights (the frozen base is not worth writing), the optimiser state, the scheduler state, the epoch, the step, and the best score so far.

Prove it live in a directory of its own: two epochs, Ctrl-C after the first `epoch 1 step` line, then `--resume` and read `resumed from ... epoch 1, step N` with the learning rate where the schedule left it.

ONNX is the serving boundary: the classifier never touches PyTorch again

- 1 Wrap the model, tokenise one short sample, `torch.onnx.export` with named inputs and outputs
- 2 Dynamic axes on batch and sequence, opset 17, `dynamo=False`
- 3 Parity on 20 held-out tickets: fp32 under 1e-3 maximum absolute logit difference
- 4 Dynamic int8 quantisation: linear weights as 8-bit integers, no calibration set, judged by task metrics, not logits

Where ONNX leaks

Custom operators, data-dependent control flow, and anything the tracer turns into a constant. Read the export warnings once.

The tokenizer is saved next to the graphs, so the runtime image never touches the Hub.

Three points of macro-F1 cost fifteen times the latency

Model	Priority macro - F1	P0 recall	Tag micro - F1	p50 ms	p95 ms	Size MB
Project 1: TF-IDF and logistic regression	0.663	0.903	n/a	1.7	2.3	6.8
Project 2: PyTorch fp32	0.693	0.742	0.541	28.0	53.3	268.0
Project 2: ONNX fp32	0.693	0.742	0.541	35.1	57.7	265.7
Project 2: ONNX int8	0.682	0.677	0.522	12.5	18.2	66.8

The subset checkpoint, six epochs, Apple silicon, test split, batch size one. Yours will differ; the shape will not.

Read the table as a customer would

P0 recall is lower

0.742 against 0.903. Project 1's threshold rule protects P0; argmax does not. The bigger model is not the safer one for the rare class.

int8 halves the latency

28.0 to 12.5 ms at p50, a quarter of the size, and it pays for it in the rare class first: P0 recall 0.742 to 0.677.

Project 1 stays in front

The capstone keeps the linear model as the cheap pre-filter and sends the transformer the tickets where three points earn their keep.

One service, two runtimes, one reason for each

```
curl -s localhost:8002/readyz
# format: model.int8.onnx

curl -s -X POST localhost:8002/classify \
  -d '{"subject":"SAML login loop for all staff",
      "body":"...", "k":3}'
```

- Tags such as SSO and Login, a priority, the score per tag above 0.05
- Three similar past tickets with their answers, each with a cosine score
- The classifier runs on ONNX Runtime; the embedder is sentence-transformers and runs on PyTorch, which is why PyTorch stays in the image

The first `make index` downloads the embedding model, about 90 MB, and takes a few minutes on CPU. The retrieval project reuses that embedder.

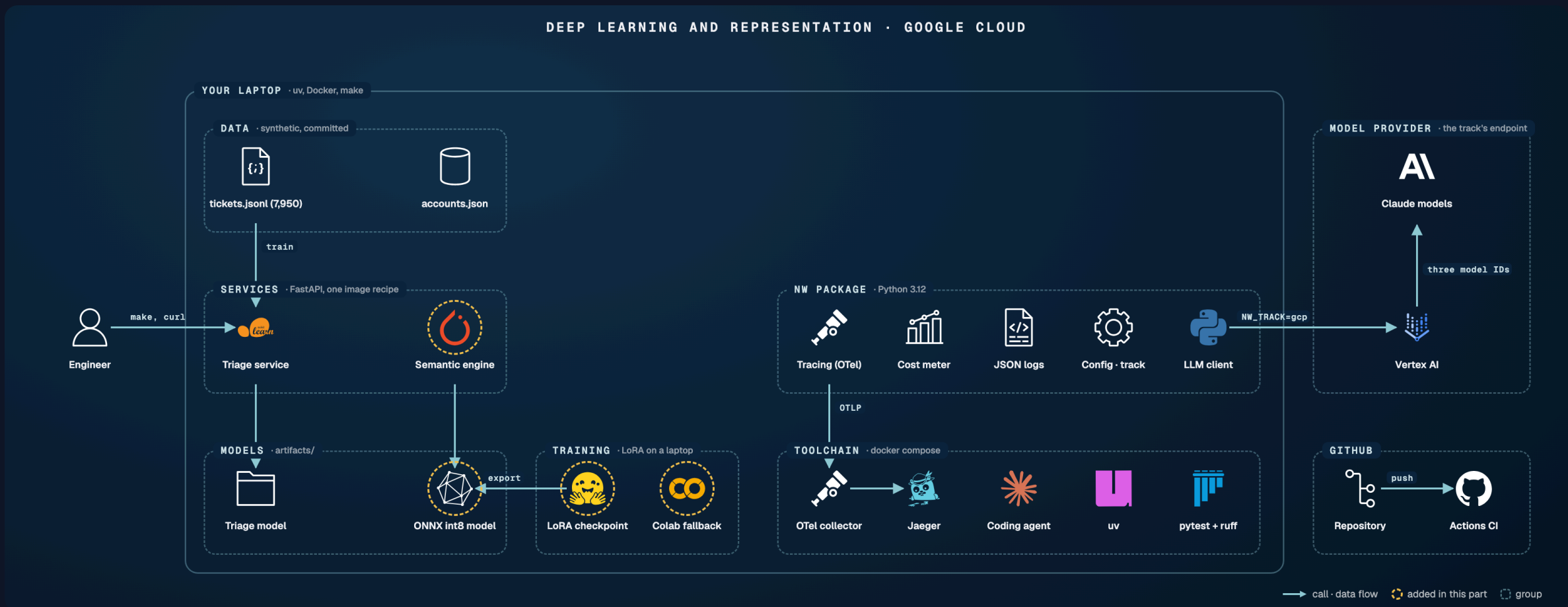
What exists after today, AWS track

DEEP LEARNING AND REPRESENTATION · AWS



The semantic engine beside the triage service, the ONNX int8 model it serves, and the LoRA checkpoint it was exported from, with Colab as the training fallback. Ringed: what this project adds.

What exists after today, Google Cloud track



Same engine, same graph, same checkpoint; Vertex AI is the endpoint. Nothing in Project 2 calls a model API.

Acceptance criteria

1. Trainable parameters under 2 percent, printed
2. Tag micro-F1 and priority macro-F1 clearly above chance on validation
3. Training resumes from a mid-run checkpoint with optimiser state
4. ONNX fp32 matches PyTorch within $1e-3$ on held-out tickets; int8 is smaller
5. The benchmark table exists with measured latency for all four rows

15 passed

what `make session03` prints when you are done

Questions worth asking

- Project 2 is slower than Project 1 by a factor you measured. For which tickets is that factor worth paying?
- The German tickets are five percent of the data and the base model is English. Where do they land in the error analysis, and what would you change?
- If the customer's stack is TensorFlow Serving, what in this session transfers unchanged?